
SAPLugin Manual – v10.0

Software Architecture 2025 – 2026

Abstract

This manual briefly explains how you can use the **SAPLugin** to automatically export your report from *Visual Paradigm* to L^AT_EX. The **SAPLugin** reads the descriptions and other meta-data in your *Visual Paradigm* model and automatically exports the structure and model content of your part 2 report to L^AT_EX. Furthermore, **SAPLugin** allows you to check your model for some common errors.

1 Set-up

This section covers the plug-in requirements and its installation.

1.1 Plug-in requirements

The **SAPLugin** requires the following Visual Paradigm version to run: Visual Paradigm 17.2 (or later). This *Visual Paradigm* version is installed in the PC classes.

Use **Help** ▸ **Update**, and then choose **Update to latest patch** at the bottom left side, to upgrade to the latest build.

1.2 Plug-in installation

Use **Help** ▸ **Install Plugin** and select the provided zip file. Restart Visual Paradigm to complete the installation.

You can install new versions of the plugin over the previous version. There is no need to uninstall the previous version.



If *Visual Paradigm* complains about a duplicate **PluginID**, it may try to install the update in a separate folder. Use **Help** ▸ **Install Plugin** to find the plugin folder and remove the other version.

2 SA project structure

SAPLugin allows you to keep your *Visual Paradigm* model structured according to a specific convention, with top-level packages for Components, Deployment nodes, Exceptions, Interactions, Interfaces, and DataTypes.

Press the “SA project structure” button to place your elements into this structure. Note that relations between your elements are retained by this operation (except some parent relationships, obviously).



If you have multiple elements with the same name, this command may need to rename some of these elements so that they can be placed under the same parent. These duplicates will then be numbered.



Note that duplicate names may already indicate a potential issue. For

example, you may have duplicated a component somewhere instead of referring to the original one.

 To avoid issues with duplicate internal *Visual Paradigm* data types, this operation will remove any internal data types that are not located under the default *Data Types* folder in *Visual Paradigm*. Otherwise, these data types can not be modified from the *Visual Paradigm* interface.

 These internal data types are not necessary at all for a successful export of your project. You can, however, always reset the default types using `Window ▸ Project Options ▸ Data Type ▸ Reset to Default ▸ OK`.

3 Checks performed by the plugin

The plugin implements some specific checks for this course, pointing to potential consistency problems or course convention violations.

→ Section 6

 Do not blindly rely on the output of the checks; they are incomplete, and there may be specific situations that are correct but flagged as a violation, or vice versa. You may report these as feedback. In any case, always use your own judgment, and only use the implemented checks to easily detect some common mistakes.

 TL;DR: Passing all checks does not mean you have a good architecture or vice versa.

A brief description of the implemented checks follows, grouped according to the categories shown in the **SAPplugin**.

 The implementation of the checks sometimes tries to take more complex situations into account than what has been described here.

3.1 General

- [Gen.01] • **Duplicate names** Two elements with the same name exist.

3.2 Components

- [Comp.01] • **Interface not provided by child component** An interface, provided by a parent component, is not provided by any of its child components.
- [Comp.02] • **Interface not required by child component** An interface, required by a parent component, is not required by any of its child components.
- [Comp.03] • **Interface not provided by parent component** An interface, provided by a child component (and not required by another child), is not provided by its parent component.
- [Comp.04] • **Interface not required by parent component** An interface, required by a child component (and not provided by another child), is not required by its parent component.
- [Comp.05] • **Component without interfaces** A component has no (required nor provided) interfaces.
- [Comp.06] • **Component not visualized** A component has no visualizations on any diagrams.
- [Comp.07] • **Interface not provided by child module** An interface, provided by a parent component, is not provided by any of its child modules.

- **Interface not required by child module** An interface, required by a parent component, is not required by any of its child modules.

3.3 Modules

- **Interface not provided by child module** An interface, provided by a parent module, is not provided by any of its child module.
- **Interface not required by child module** An interface, required by a parent module, is not required by any of its child module.
- **Interface not provided by parent module** An interface, provided by a child module (and not required by another child), is not provided by its parent module.
- **Interface not required by parent module** An interface, required by a child module (and not provided by another child), is not required by its parent module.
- **Module without interfaces** A module has no (required nor provided) interfaces.
- **Module with component children** A module has components as children.
- **Module not visualized** A module has no visualizations on any diagrams.
- **Interface not provided by parent component** An interface, provided by a child module (and not required by another child), is not provided by its parent component.
- **Interface not required by parent component** An interface, required by a child module (and not provided by another child), is not required by its parent component.

3.4 Interfaces

- **Interface with association** An interface in your model is connected to another element through an association.
- **Interface never provided** An interface in your model is not provided by any component or module.
- **Interface never required** An interface in your model is not required by any component or module.
- **Interface not visualized** An interface has no visualizations on any diagrams.

3.5 Operations

- **Interface without operations** An interface has no operations.
- **Operation outside interface** An operation has been defined on an element that is not an interface.
- **Missing data types** An operation refers to undefined data types.
- **Missing exception types** An operation refers to undefined exception types.

3.6 Component Diagrams

- | | |
|----------------|---|
| [Diag.comp.01] | • Missing C/S context diagram The project does not contain a C/S context diagram. Make sure its name is correctly specified so that SAPPlugin can find it. |
| [Diag.comp.02] | • Missing C/S primary diagram The project does not contain a primary C/S diagram. Make sure its name is correctly specified so that SAPPlugin can find it. |
| [Diag.comp.03] | • Missing C/S diagrams The project does not contain any C/S diagram. |
| [Diag.comp.04] | • Empty component diagram You have a completely empty component diagram. |
| [Diag.comp.05] | • Interface dangling on context diagram The interface on this context diagram does not have a providing or requiring component. This may indicate there is a connecting element that requires or provides this interface, but that element is not connected or not visible on the diagram. |
| [Diag.comp.06] | • Component missing on context diagram The component has no parent (indicating it is the system itself or one of the external services with which it interacts), but the component does not seem to be visualized on the C/S context diagram. |

3.7 Module Diagrams

- | | |
|---------------|--|
| [Diag.mod.01] | • Missing module context diagram The project does not contain a module context diagram. Make sure its name is correctly specified so that SAPPlugin can find it. |
| [Diag.mod.02] | • Missing module diagrams The project does not contain any module diagram. |
| [Diag.mod.03] | • Empty module diagram You have a completely empty module diagram. |

3.8 Sequence Diagrams

- | | |
|---------------|--|
| [Diag.seq.01] | • Inconsistent sequence diagram call The operation call on a sequence diagram is not possible according to the required interfaces of the calling component. |
| [Diag.seq.02] | • Empty sequence diagram You have a completely empty sequence diagram. |
| [Diag.seq.03] | • Non-leaf components The sequence diagram uses components that have sub-components. This hides details of how the functionality is realized by those sub-components. |
| [Diag.seq.04] | • Mixing components and modules The sequence diagram uses both components and modules in the same diagram. |

3.9 Deployment Diagrams

- | | |
|----------------|--|
| [Diag.depl.01] | • Component part of node A component has a deployment node as its parent. |
| [Diag.depl.02] | • Module part of node A module has a deployment node as its parent. |

[Diag.depl.03]	• Component never deployed A component (or any of its descendants) has not been deployed on some node.
[Diag.depl.04]	• Component and ancestor both deployed A component and one of its ancestors have both been deployed.
[Diag.depl.05]	• Interacting components cannot communicate A component with a required interface cannot communicate with any component that provides that interface according to the deployment links.
[Diag.depl.06]	• Unnecessary deployment connection Two nodes are connected by a link, but there is no component pair that can use that link to communicate.
[Diag.depl.07]	• Missing deployment context diagram The project does not contain a deployment context diagram. Make sure its name is correctly specified so that <code>SAPPlugin</code> can find it.
[Diag.depl.08]	• Missing deployment diagrams The project does not contain any deployment diagram.
[Diag.depl.09]	• Empty deployment diagram You have a completely empty deployment diagram.

3.10 Descriptions

[Gen.desc.01]	• Missing component responsibility description A component does not have a description of its responsibility.
[Gen.desc.02]	• Missing module responsibility description A module does not have a description of its responsibility.
[Gen.desc.03]	• Missing effect description An operation does not have a description of its effect.
[Gen.desc.04]	• Missing datatype description A datatype does not have a description.
[Gen.desc.05]	• Missing return type description An operation specifies a return type but it does not have a description.
[Gen.desc.06]	• Missing node description A node lacks a description.



The required and provided interface checks look for certain types of relations between the interfaces and the components. The provided check relies on the *realization* relation, while the required check relies on the *usage* relation. If you use generic associations in your model, the checks relying on these types of relations will fail.

4 Structure of your Visual Paradigm project

Automatically exporting your L^AT_EX report from *Visual Paradigm* requires you to maintain a sound model of your architecture. This section elaborates on how `SAPPlugin` reads and interprets your model in order to be able to properly export it.

4.1 Exporting Component Descriptions

The super- and sub-component relations need to be explicitly in your model. Verify that, in your model explorer, components are correctly nested corresponding to their super- or sub-component relationships.

→ Subsection 4.3

This export functionality automatically includes a reference to all interfaces that are required or provided by a component.



If you duplicate interfaces on different diagrams, they will be listed multiple times here. This happens because, according to your model, your component provides several interfaces with the same name. To avoid this, drag the existing interface from the model explorer to the other diagrams (to create an auxiliary view).

Responsibility

The description of the responsibilities of a component are extracted from the component's description (on the *General* tab of its specification). All names of known components that appear in this description (case-sensitive) will be replaced automatically by references to that component.

4.2 Exporting Modules

When exporting modules, the **SAPLugin** will automatically list the modules contained in your project. Modules are represented by components that are stereotyped with the `<<module>>` stereotype.

→ Subsection 4.3

This export functionality automatically includes a reference to all interfaces that are required or provided by a module.

Responsibility

The description of the responsibilities of a module are extracted from the module's description (on the *General* tab of its specification). All names of known module that appear in this description (case-sensitive) will be replaced automatically by references to that module.

4.3 Exporting Interfaces and their Operations

When exporting interfaces, the **SAPLugin** will create a list of all the interfaces in your project, and for each interface, list the operations of that interface.

To properly extract the operations, it is necessary for your project to define all operations for each interface.



Make sure that you use valid types for the parameter and return types in your operations. Every type used in your operations should be a valid Data Type.

→ subsection 4.5

If an invalid type is specified, the exported catalog will show: **Invalid type:**
[Type (e.g., com.vp.plugin.model.IComponent)] ElementName .

Effect

The effect of an operation is extracted from the operations description (on the *General* tab of its specification).

4.4 Exporting Exceptions

Visual Paradigm does not have a separate model element for representing exceptions. **SAPLugin** automatically determines if a class is an exception, when it is listed in the *Raised Exceptions* tab of an operation. When the exception is listed there, it will also be automatically included in the *throws* clause of that operation's signature.



The **SAPLugin** will attempt to derive whether a class is an exception by checking inheritance hierarchy. This means that you base exception class will also be listed with the exceptions if you have modeled the inheritance relations correctly.



Exception classes that are not raised anywhere and that are not linked

(through inheritance) with other exception classes will be listed with the Data Types.

4.5 Exporting Data Types

Make a class for all data types you use. These will be collected by the **SAPPlugin** and listed in the *DataTypes* section of the element catalog.

Primitive types A number of primitives are supported by default. You can use these anywhere you can use your own data types. There are the following:

- boolean
- byte
- char
- double
- float
- int
- long
- short
- string
- void

Container types The following container types are recognized by the **SAPPlugin**, so you do not need to define a class for them:

- List
- Map
- Set
- Tuple

Use generics (< >) to specify the types they contain, just as you would in Java. **SAPPlugin** will recognize these types as well. Furthermore, you can also use any self-defined class, if desired.

Previously defined container types with square brackets [] will continue to work as before. But, you may experience issues when using them for entering the return type or attribute type directly on a class diagram.¹



These container types are only recognized if you also specify the type they contain.



SAPPlugin does not check the number of type arguments to include for each container type (i.e., one for List and Set, and two for Map).

Example `Map<SomeClass, Boolean>`

Description The description of a data type is extracted from the data type's description (on the *General* tab of its specification.)

4.5.1 Data type attributes

You can specify attributes of your data types. These will also be included in the export. Limit yourself to specifying attributes that are relevant with regard to realizing your quality requirements.



The types of these attributes need to exist as well.

4.6 Missing elements

DataTypes Missing elements such as non-existing data types will be output by the tool as follows: `UndefinedDataType`.

Descriptions Missing descriptions will be marked by the tool as `Undefined`.



Make sure to check your final report for missing elements!

¹ Visual Paradigm interprets this as a type modifier

4.7 Descriptions and responsibilities

All names of known components that appear in descriptions (case-sensitive) will be replaced automatically by references to that component.

It is possible that you want to describe multiple components and still include a reference. If you use the plural form ‘as-is’ the name will not match and will not be replaced by a reference. If you do want it to be replaced by a reference, use ‘|’ to append the suffix. E.g., Gateway|s will match and will be replaced with a reference and the suffix: Gateways.

 *Avoid having multiple components with the same name.*

If you have multiple components with the same name, the plugin cannot replace their names with references in the export. If you do want to add references, you will have to use the fully qualified names of the components (e.g., `Components.Some.SubComponent` and `Components.Another.SubComponent`) everywhere.

 *Avoid using LaTeX specific characters.*

The `SAPLugin` will try to escape many different characters with special meaning in LaTeX (e.g., `#$%&{\}~`). However, it is preferable to avoid these characters in the names of your elements.

5 Exporting to L^AT_EX Report

You can use the plug-in via the plug-in tab in Visual Paradigm. Go to *Plugin* ▸ *Export to L^AT_EX Report*. Next, you will be asked to provide the location of the output files.

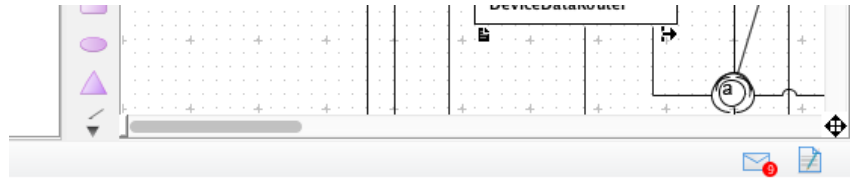
The plugin will export to the following files:

- `report.tex` The main structure of your report. *If you export to the report again, this file is **never overwritten**; in that case, `report-empty.tex` is used (and overwritten!) instead.*
- `rationale.tex` The part of your report in which you can write the rationale. *If you export to the report again, this file is **never overwritten**; in that case, `rationale-empty.tex` is used (and overwritten!) instead.*
- `diagrams.tex` and files in `images/` An appendix with all diagrams in your *Visual Paradigm* project. *If you export to the report again, these files are **always overwritten**.* You can change the sorting order of the diagrams by prefixing them with `[sortingkey]` (E.g., `[A]Decomposition of Component X`). The `SAPLugin` will sort the diagrams alphabetically using the `sortingkeys`. They will not be included in the caption names of the diagrams.
- `catalog.tex` An appendix with the element catalog from your *Visual Paradigm* project. *If you export to the report again, these files are **always overwritten**.*
- `sareport.cls` The `sareport` documentclass file.
- `KULeuven.pdf` The KU Leuven logo for the report.

 Because files can be easily overwritten, we recommend using a version control system for your L^AT_EX and image files to easily back them up and restore them if necessary.

 Note the messages pane in the bottom-right corner in *Visual Paradigm*

and expand it if needed. `SAPplugin` will notify you when the export has finished.



5.1 References

The `SAPplugin` automatically creates labels for several model elements and diagrams so you can refer to them in your main report text (i.e., `rationale.tex`).

i Note that you can always look at the exported `.tex`-files for looking up a certain label.

If something is defined as a label in $\text{\LaTeX}$ (`\label{labelname}`), you can refer to it in multiple ways:

- Refer to the number: `\ref{labelname}`
- Refer to the page: `\pageref{labelname}`
- Refer to the name: `\nameref{labelname}`

i Do use these labels! In combination with `hyperref`, they provide clickable references in the PDF which makes it much easier to navigate your final report.

5.1.1 Model elements

The following schema is used:

Components : `comp:<fullyQualifiedNameWithoutPeriodsOrSpaces>`

Example: A component named *CompA* under super-component *CompSuper* in the *Components*-package will get the label:
`comp:ComponentsCompSuperCompA.`

Interfaces : `int:<fullyQualifiedNameWithoutPeriodsOrSpaces>`

Example: An interface named *InterfA* in the *Interfaces*-package will get the label:
`int:InterfacesInterfA.`

Exceptions : `ex:<fullyQualifiedNameWithoutPeriodsOrSpaces>`

Example: An exception named *IllegalArgumentException* in the *Exceptions*-package will get the label:
`ex:ExceptionsIllegalArgumentException.`

DataTypes : `data:<fullyQualifiedNameWithoutPeriodsOrSpaces>`

Example: A datatype named *SomeData* in the *Datatypes*-package will get the label:
`data:DatatypesSomeData.`

5.1.2 Diagrams

The following naming scheme is used for creating diagram labels:

`diag:<diagramType>:<diagramNameWithoutPeriodsOrSpaces>`

Examples:

- Component diag:Component:cscontext
- Deployment diag:Deployment:deplcontext
- Sequence diag:Interaction:SomeSequence

6 Feedback and issues

If you run into a problem with **SAPugin**, or if you have a feature request or other feedback to share with us, contact us at:

SoftwareArchitecture@cs.kuleuven.be

In case of a problem, include at least the following information:

- Description of the problem and steps to reproduce. What were you doing, and what went wrong?
- Your *Visual Paradigm* project (or a minimal example) which allows us to reproduce the problem
- *Visual Paradigm* version and build number
- **SAPugin** version
- Log file with the exceptions (About ▷ Export Log File).